
Priority Scheduling

Operating Systems and Systems Programming Project

Section: 6

Team No.: 20

Team Members:

Somisetty Ketana Sindhu-2520030487

Yerrolaa Eekshitha— 2520030615

Poojitha Rani— 2520030460

Abstract

- A C-based, Linux/Ubuntu terminal-oriented CPU Scheduling project.
- Implements **Non-Preemptive Priority Scheduling**.
- Implements **Preemptive Priority Scheduling**.
- Takes Process ID, Arrival Time, Burst Time and Priority as input.
- Calculates Completion Time, Turnaround Time, Waiting Time and Response Time.
- Generates Gantt Charts to represent process execution.
- Demonstrates Linux system calls, process management and IPC concepts.

Introduction

- Linux-based CPU Scheduling project developed using C.
- Implements Priority Scheduling for multiple processes.
- Provides:
 - Process Input
 - Non-Preemptive Scheduling
 - Preemptive Scheduling
 - Performance Calculation
 - Gantt Chart
- Uses Arrival Time, Burst Time and Priority.
- Smaller priority number represents higher priority.



Results – CO1

LINUX SYSTEM CALLS

Implemented basic Linux system calls.

Demonstrated:

getpid()
getppid()
getuid()
uname()
write()

Demonstrated user-space and kernel interaction.

Implemented file and system services using:

open()
write()
close()
getcwd()

Demonstrated shell command execution using system().

```
katana_sirisha@katana:/net:/Users/katana_sirisha/Downloads/文档/NO. 1000000000 and YEAR/OS/PROJECT/SRC/CO1_SystemCalls$ gcc system_calls_demo.c -o system_calls_demo
katana_sirisha@katana:/net:/Users/katana_sirisha/Downloads/文档/NO. 1000000000 and YEAR/OS/PROJECT/SRC/CO1_SystemCalls$ ./system_calls_demo

=====
CO-1: LINUX SYSTEM CALLS
=====

[1] Process ID (PID): 266
[2] Parent Process ID (PPID): 264
[3] User ID (UID): 1000

[4] Linux System Information
System : Linux
Node : katana
Release : 6.16.33.2-microsoft-standard-66.2
Version : #1 SMP PREEMPT_DYNAMIC Thu Jan 18 21:54:43 UTC 2024
Machine : x86_64

[5] write() system call executed successfully.

=====
CO-1 DEMO COMPLETED
=====
katana_sirisha@katana:/net:/Users/katana_sirisha/Downloads/文档/NO. 1000000000 and YEAR/OS/PROJECT/SRC/CO1_SystemCalls$
```

```
PS C:\Users\katana_sirisha\OneDrive\文档\NO. 1000000000 and YEAR\OS\PROJECT> cd SRC\CO1_SystemCalls
PS C:\Users\katana_sirisha\OneDrive\文档\NO. 1000000000 and YEAR\OS\PROJECT\SRC\CO1_SystemCalls> gcc user_kernel_demo.c -o user_kernel_demo
PS C:\Users\katana_sirisha\OneDrive\文档\NO. 1000000000 and YEAR\OS\PROJECT\SRC\CO1_SystemCalls> ./user_kernel_demo

=====
CO-2: USER SPACE & KERNEL SPACE
=====

1. Program is currently executing in User Space.
2. Requesting information from the linux kernel...
3. System call: getpid()
4. Kernel returned the Process ID: 26664

-----
User Space -> System Call -> Kernel
-----

The program requested an OS service through the system call interface.

=====
CO-2 STEP 2 COMPLETED
=====
PS C:\Users\katana_sirisha\OneDrive\文档\NO. 1000000000 and YEAR\OS\PROJECT\SRC\CO1_SystemCalls>
```

Results – CO1

```
PS C:\Users\ketana.sindhu\OneDrive\文档\KL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO1_SystemCalls> gcc system_call_services.c -o system_call_services
ces
PS C:\Users\ketana.sindhu\OneDrive\文档\KL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO1_SystemCalls> ./system_call_services

=====
CO-1: SYSTEM CALL SERVICES
=====

[1] Process Service
    getpid() -> Current PID: 35384

[2] File Service
    open() -> File opened successfully
    write() -> 31 bytes written
    close() -> File closed successfully

[3] Process Environment
    Current Working Directory:
    C:\Users\ketana.sindhu\OneDrive\文档\KL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO1_SystemCalls

=====
SYSTEM CALL DEMONSTRATION COMPLETE
=====

PS C:\Users\ketana.sindhu\OneDrive\文档\KL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO1_SystemCalls>
```

```
PS C:\Users\ketana.sindhu\OneDrive\文档\KL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO1_SystemCalls> gcc shell_command_demo.c -o shell_command_demo
PS C:\Users\ketana.sindhu\OneDrive\文档\KL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO1_SystemCalls> ./shell_command_demo

=====
CO-1: SHELL & COMMAND EXECUTION
=====

[1] Current C Program
    PID: 28328

[2] Executing Linux command: ls -l

'ls' is not recognized as an internal or external command,
operable program or batch file.

[3] Linux command execution completed.

Command Execution Journey:
=====
C Program
  |
  v
system()
  |
  v
Shell
  |
  v
Linux Command (ls -l)
  |
  v
Kernel Services
=====

CO-1 STEP 4 COMPLETE
=====

PS C:\Users\ketana.sindhu\OneDrive\文档\KL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO1_SystemCalls>
```

Results – CO2

PROCESS MANAGEMENT

PROCESS CREATION

Implemented process creation using fork().

Created parent and child processes.

PROCESS EXECUTION

Implemented exec() / execl().

Demonstrated program replacement.

PROCESS SYNCHRONIZATION

Used wait() for parent-child synchronization.

PROCESS LIFECYCLE

Demonstrated:

Created

Ready

Running

Terminated

```
root@kali:~/Desktop/CO2# ./process_creation.c
=====
ID-1: PROCESS CREATION
=====

[Parent] Parent process started.
[Parent] Parent PID: 1000

[Parent] fork() created child process.
[Parent] Child PID: 1001
[Parent] Waiting for child process...
[Child] Child process created successfully.
[Child] Child PID: 1001
[Child] Parent PID: 1000
[Child] Child process is running...
[Child] Child process completed.
[Parent] Child process has completed.
[Parent] Parent process continues execution.

=====
PROCESS CREATION COMPLETE
=====

root@kali:~/Desktop/CO2#
```

```
root@kali:~/Desktop/CO2# ./process_creation.c
=====
ID-1: FORK + EXEC
=====

[Parent] Parent process started.
[Parent] Parent PID: 1342

[Parent] Child PID: 1343
[Parent] Waiting for child...
[Child] Child process created.
[Child] Child PID: 1343
[Child] Counting "ls -l" using wait().

total 40
-rw-rw-r-- 1 ketan.ketan ketan 1024 Sep 21 21:40 process_creation.c
-rw-rw-r-- 1 ketan.ketan ketan 1024 Sep 21 21:40 process_creation.o
-rw-rw-r-- 1 ketan.ketan ketan 1024 Sep 21 21:41 process.o
-rw-rw-r-- 1 ketan.ketan ketan 1315 Sep 21 21:41 process.o.o.f

[Parent] Child completed.
[Parent] Parent continues execution.

=====
FORK + EXEC COMPLETE
=====

root@kali:~/Desktop/CO2#
```

Results – CO2

```
ketana_sindhu@ketalone: /mnt/c/Users/ketana_sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECTS cd SRC/CO2_ProcessScheduling
ketana_sindhu@ketalone: /mnt/c/Users/ketana_sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO2_ProcessScheduling$ gcc process_life
cycle.c -o process_lifecycle
ketana_sindhu@ketalone: /mnt/c/Users/ketana_sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO2_ProcessScheduling$ ./process_lifec
ycle
```

CO-2: PROCESS LIFECYCLE

```
[Parent] Process is CREATED.
[Parent] PID: 1268

[Parent] Child process created.
[Parent] Child PID: 1269
[Parent] Waiting for child termination...
[child] Process CREATED.
[child] PID: 1269
[child] State: READY
[child] State: RUNNING
[child] Performing some work...
[child] Work completed.
[child] State: TERMINATED

[Parent] Child process has TERMINATED.
[Parent] Parent continues execution.
```

PROCESS LIFECYCLE COMPLETE

```
ketana_sindhu@ketalone: /mnt/c/Users/ketana_sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO2_ProcessScheduling$
```

```
ketana_sindhu@ketalone: /mnt/c/Users/ketana_sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO2_ProcessScheduling$ gcc process_management.c -o process_management
ketana_sindhu@ketalone: /mnt/c/Users/ketana_sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO2_ProcessScheduling$ ./process_management
```

CO-2: PROCESS MANAGEMENT

Enter number of processes (1-30): 2

Process 1

PID : 1
Arrival Time : 3
Burst Time : 4
Priority : 6

Process 2

PID : 2
Arrival Time : 6
Burst Time : 9
Priority : 3

PROCESS TABLE

PID	AT	BT	Priority	Remaining	State
1	3	4	6	4	READY
2	6	9	3	9	READY

Process management structure created successfully.

Each process currently contains:

- Process ID
- Arrival Time
- Burst Time
- Priority
- Remaining time
- Scheduling metrics
- Process state

PROCESS MANAGEMENT COMPLETE

```
ketana_sindhu@ketalone: /mnt/c/Users/ketana_sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO2_ProcessScheduling$
```

Results – CO2

NON-PREEMPTIVE PRIORITY SCHEDULING INPUT

Process ID

Arrival Time

Burst Time

Priority

PROCESSING

Checks processes that have arrived.

Selects the highest-priority process.

Executes the selected process until completion.

Moves to the next eligible process.

OUTPUT

Completion Time

Turnaround Time

Waiting Time

Response Time

Gantt Chart

```
betars_cslind@betarsone: /mnt/c/users/betars_cslind/OneDrive/2025/01/UNIVERSITY_2nd_YEAR/CS/PROJECTS/CO2_ProcessScheduling$ ./priority_non_preemptive.py
betars_cslind@betarsone: /mnt/c/users/betars_cslind/OneDrive/2025/01/UNIVERSITY_2nd_YEAR/CS/PROJECTS/CO2_ProcessScheduling$ ./priority_non_preemptive.py

=====
NON-PREEMPTIVE PRIORITY SCHEDULING
=====

Priority Rule:
Smaller priority number = higher priority

Enter number of processes (1-10): 4

Process 1:
PID: 1
Arrival Time: 0
Burst Time: 5
Priority: 1

Process 2:
PID: 2
Arrival Time: 1
Burst Time: 4
Priority: 3

Process 3:
PID: 3
Arrival Time: 2
Burst Time: 4
Priority: 2

Process 4:
PID: 4
Arrival Time: 4
Burst Time: 2
Priority: 1

=====
PROCESS INFORMATION
=====

PID  AT  BT  Priority
1  0  5  1
2  1  4  3
3  2  4  2
4  4  2  1
```

```
=====
SCHEDULING RESULTS
=====

PID  AT  BT  Priority  CT  TAT  WT  RT
1  0  5  1  5  5  0  0
2  1  4  3  9  7  0  4
3  2  4  2  6  4  0  0
4  4  2  1  6  2  0  0

Average Waiting Time: 0.00
Average Turnaround Time: 2.25
Average Response Time: 0.00

=====
GANTT CHART
=====

| P1 | P3 | P2 | P4 |
|-----|
| 0   5   6   10  14 |

=====
NON-PREEMPTIVE SCHEDULING COMPLETE
=====

betars_cslind@betarsone: /mnt/c/users/betars_cslind/OneDrive/2025/01/UNIVERSITY_2nd_YEAR/CS/PROJECTS/CO2_ProcessScheduling$
```

Results – CO2

PREEMPTIVE PRIORITY SCHEDULING PROCESSING

Checks available processes at every time unit.

Selects the highest-priority process.

Allows higher-priority processes to preempt the running process.

Maintains remaining burst time.

Resumes preempted processes when selected again.

OUTPUT

Completion Time

Turnaround Time

Waiting Time

Response Time

Gantt Chart

```
PS C:\Users\ketana.sindhu\OneDrive\文檔\ML UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO2_ProcessScheduling> .\priority_preemptive.c -n priority_pre
PS C:\Users\ketana.sindhu\OneDrive\文檔\ML UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO2_ProcessScheduling> gcc priority_preemptive.c -o priority_pre
PS C:\Users\ketana.sindhu\OneDrive\文檔\ML UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO2_ProcessScheduling> ./priority_preemptive

=====
PREEMPTIVE PRIORITY SCHEDULING
=====

Priority Rule:
Smaller priority number = higher priority

Enter number of processes (1-20): 3

Process 1:
PID      : 1
Arrival Time : 0
Burst Time : 8
Priority   : 3

Process 2:
PID      : 2
Arrival Time : 2
Burst Time : 3
Priority   : 1

Process 3:
PID      : 3
Arrival Time : 4
Burst Time : 2
Priority   : 2

=====
PROCESS INFORMATION
=====

=====
SCHEDULING RESULTS
=====

=====
PROCESS INFORMATION
=====


| PID | AT | BT | Priority |
|-----|----|----|----------|
| 1   | 0  | 8  | 3        |
| 2   | 2  | 3  | 1        |
| 3   | 4  | 2  | 2        |


=====

=====
SCHEDULING RESULTS
=====


| PID | AT | BT | Priority | CT | TAT | WT | RT |
|-----|----|----|----------|----|-----|----|----|
| 1   | 0  | 8  | 3        | 13 | 13  | 5  | 0  |
| 2   | 2  | 3  | 1        | 5  | 3   | 0  | 0  |
| 3   | 4  | 2  | 2        | 7  | 3   | 1  | 1  |


=====

Average waiting Time : 2.00
Average Turnaround Time : 6.33
Average Response Time : 0.33

=====
GANTT CHART
=====

| P1 | P2 | P3 | P1 |
| 0  | 2  | 5  | 7  | 13

=====
PREEMPTIVE SCHEDULING COMPLETE
=====

PS C:\Users\ketana.sindhu\OneDrive\文檔\ML UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO2_ProcessScheduling>
```

Results – CO2

```
PS C:\Users\ketana.sindhu\OneDrive\文档\VL UNIVERSITY 2nd YEAR\OS\PROJECT> cd SRC\CO3_IPC
PS C:\Users\ketana.sindhu\OneDrive\文档\VL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO3_IPC> vs1
ketana_sindhu@ketaZone: /mnt/c/Users/ketana.sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO3_IPC$ gcc signal_demo.c -o signal_demo
ketana_sindhu@ketaZone: /mnt/c/Users/ketana.sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO3_IPC$ ./signal_demo

=====
CO-3: SIGNAL IPC DEMO
=====

[Parent] Parent PID: 2420
[Parent] Child PID: 2421
[Child] Child process created.
[Child] Child PID: 2421
[Child] Waiting for SIGUSR1...

[Parent] Sending SIGUSR1 to child...
[Parent] SIGUSR1 sent successfully.
[Child] SIGUSR1 received! Signal handler executed.
[Child] Signal handling completed.
[Parent] Child process completed.

=====
SIGNAL DEMO COMPLETE
=====

ketana_sindhu@ketaZone: /mnt/c/Users/ketana.sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO3_IPC$
```

```
PS C:\Users\ketana.sindhu\OneDrive\文档\VL UNIVERSITY 2nd YEAR\OS\PROJECT> cd SRC\CO3_IPC
PS C:\Users\ketana.sindhu\OneDrive\文档\VL UNIVERSITY 2nd YEAR\OS\PROJECT\SRC\CO3_IPC> vs1
ketana_sindhu@ketaZone: /mnt/c/Users/ketana.sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO3_IPC$ gcc shared_memory.c -o shared_memory
ketana_sindhu@ketaZone: /mnt/c/Users/ketana.sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO3_IPC$ ./shared_memory

=====
CO-3: SHARED MEMORY IPC
=====

[System] Shared memory created.
[System] Shared Memory ID: 0
[System] Shared memory attached.

[Parent] Parent process started.
[Parent] PID: 2507
[Parent] Child PID: 2508
[Parent] Data written to shared memory.

[Child] Child process started.
[Child] PID: 2508
[Child] Reading shared memory...
[Child] Data received:
"Priority Scheduling data stored in Shared Memory."
[Child] Shared memory detached.
[Parent] Child process completed.
[Parent] Shared memory detached.
[Parent] Shared memory removed.

=====
SHARED MEMORY COMPLETE
=====

ketana_sindhu@ketaZone: /mnt/c/Users/ketana.sindhu/OneDrive/文档/VL UNIVERSITY 2nd YEAR/OS/PROJECT/SRC/CO3_IPC$
```

HARDWARE & SOFTWARE

HARDWARE

- Computer / Laptop
- Keyboard
- Display

SOFTWARE

- Operating System: Linux / Ubuntu / WSL2
- Programming Language: C
- Compiler: GCC
- Interface: Linux Terminal
- Editor: Visual Studio Code
- System Programming: Linux/POSIX APIs
- Version Control: Git / GitHub

Conclusion

- Developed a Linux-based Priority Scheduling project using C.
- Implemented **Non-Preemptive Priority Scheduling**.
- Implemented **Preemptive Priority Scheduling**.
- Calculated CT, TAT, WT and RT.
- Generated Gantt Charts for process execution.
- Demonstrated CO-1 — Linux system calls.
- Demonstrated CO-2 — Process creation and process management.
- Demonstrated CO-3 — Inter-Process Communication.
- Successfully tested all implemented modules.

Future Scope

SCHEDULING EXTENSION

- Implement additional CPU scheduling algorithms.
- Compare scheduling performance.
- Support larger process sets.

VISUALIZATION

- Develop interactive Gantt Charts.
- Add graphical scheduling interface.
- Provide real-time process visualization.

SYSTEM EXTENSION

- Integrate additional Linux/POSIX APIs.
- Extend process monitoring.
- Expand IPC-based communication

Thank you

Priority Scheduling — Team 20 | Section - 6

Somisetty Ketana Sindhu-2520030487

Yerrolaa Eekshitha— 2520030615

Poojitha Rani— 2520030460
